

BarberTime — Entrega Sprint 2 (Integração com MOM)

Disciplina: LDAMD — PUC Minas
Projeto: BarberTime — agendamento em barbearia
Sprint: 2 — Integração com Middleware Orientado a Mensagens
Aluno: Caio Kodato
Data da demonstração: 25/05/2026

Objetivo da sprint

Integrar um **Middleware Orientado a Mensagens (MOM)** ao backend, com comunicação **assíncrona orientada a eventos**, conforme o Projeto Integrador LDAMD.

Fluxo demonstrado:

- 1. Cliente solicita agendamento → status `pending` → evento `appointment.requested` → notificação ao **barbeiro**.
- 2. Barbeiro confirma → status `confirmed` → evento `appointment.confirmed` → notificação ao **cliente**.

Entregas obrigatórias e evidências

Entrega exigida	Evidência no repositório
MOM configurado e operacional	<code>docker-compose.yml</code> , Figuras 3 e 4
Produtor e consumidor implementados	Figuras 1 e 2; <code>src/messaging/</code> , <code>src/workers/</code>
Documentação dos eventos	<code>EVENTOS_MOM.pdf</code>
Demonstração de assincronicidade	Figura 2 (worker em processo separado da API)
Relatório de integração (1 página)	<code>RELATORIO_INTEGRACAO_MOM.pdf</code>

Critérios de avaliação (20 pontos)

Critério	Peso	Atendido	Evidência
MOM funcionando corretamente	25%	Sim	Figuras 3 e 4; broker RabbitMQ 3.13
Produtor e consumidor de eventos	30%	Sim	Figuras 1 e 2; eventos <code>requested</code> e <code>confirmed</code>
Documentação dos eventos	20%	Sim	<code>EVENTOS_MOM.pdf</code>

Assincronicidade real no fluxo	15%	Sim	Worker sem chamada REST ao produtor
Clareza do relatório de integração	10%	Sim	RELATORIO_INTEGRACAO_MOM.pdf

Arquitetura resumida

Papel	Componente	Tecnologia
Produtor	API Express	EventPublisher após persistência no PostgreSQL
Broker	RabbitMQ	Exchange topic barbertime.events
Consumidor	Notification Worker	npm run worker:notifications
Persistência	PostgreSQL	Agendamentos pending / confirmed

Agendamento demonstrado: 24f21de9-f6f5-428e-b45a-ccf50eb2b364 — cliente Caio Kodato Cliente, barbeiro João Barbeiro, 27/06/2026 14:30–15:00 (America/Sao_Paulo).

Evidências visuais

Figura 1 — Produtor (API REST)

```
Restarting 'src/server.js'
[RabbitMQ] Conectado e topologia pronta.
BarberTime API ouvindo na porta 3000
[RabbitMQ] Evento publicado: appointment.requested (e44bb586-b6fd-4f3c-ada0-d1c53e511f21)
[RabbitMQ] Evento publicado: appointment.confirmed (7c10eec4-b808-42e3-acdb-b3ca828a452f)
Restarting 'src/server.js'
[RabbitMQ] Conectado e topologia pronta.
Restarting 'src/server.js'
[RabbitMQ] Conectado e topologia pronta.
Restarting 'src/server.js'
Restarting 'src/server.js'
Restarting 'src/server.js'
Restarting 'src/server.js'
Restarting 'src/server.js'
Restarting 'src/server.js'
Restarting 'src/server.js'
Restarting 'src/server.js'
Restarting 'src/server.js'
Restarting 'src/server.js'
[RabbitMQ] Conectado e topologia pronta.
Restarting 'src/server.js'
[RabbitMQ] Conectado e topologia pronta.
BarberTime API ouvindo na porta 3000
Restarting 'src/server.js'
Restarting 'src/server.js'
[RabbitMQ] Conectado e topologia pronta.
BarberTime API ouvindo na porta 3000
[RabbitMQ] Evento publicado: appointment.requested (e44bb586-b6fd-4f3c-ada0-d1c53e511f21)
[RabbitMQ] Evento publicado: appointment.confirmed (7c10eec4-b808-42e3-acdb-b3ca828a452f)
█
```

A API registra a publicação de `appointment.requested` e `appointment.confirmed` após as operações REST.

Figura 2 — Consumidor (Notification Worker)

```

caiov@Kodato MINGW64 /c/RepositoriosPUC/BarberTime (main)
$ npm run worker:notifications

> barbertime-api@1.0.0 worker:notifications
> node src/workers/notificationWorker.js

[RabbitMQ] Conectado e topologia pronta.
[NotificationConsumer] Ouvindo filas:
- barbertime.notifications.barber
- barbertime.notifications.client
[Worker] Notification worker em execução. Aguardando eventos...

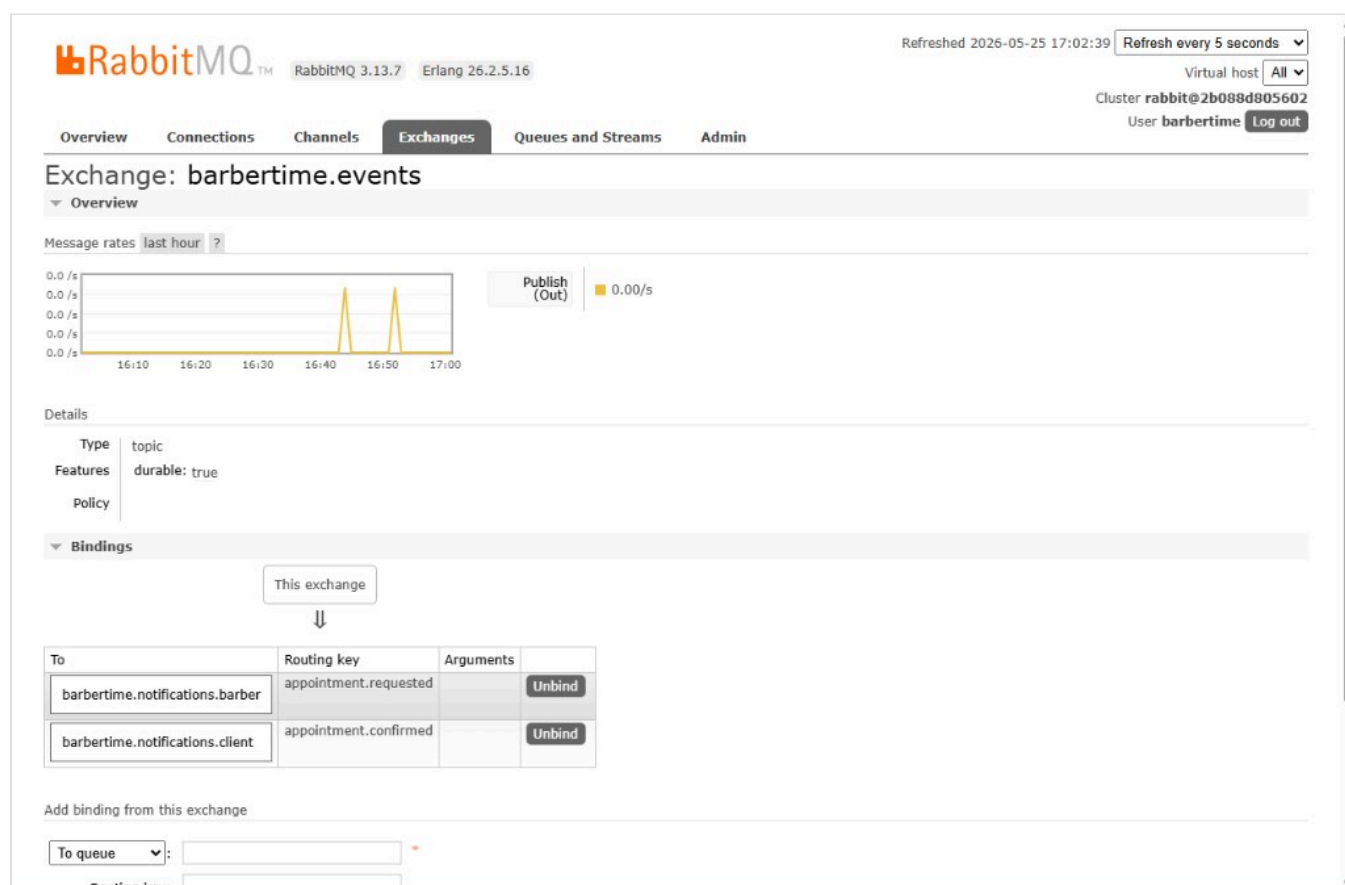
[NOTIFICAÇÃO → BARBEIRO] Nova solicitação de agendamento
Evento:    appointment.requested (e44bb586-b6fd-4f3c-ada0-d1c53e511f21)
Barbeiro:  João Barbeiro <joao.barbeiro@barbertime.seed>
Cliente:   Caio Kodato Cliente
Horário:   2026-06-27T17:30:00.000Z → 2026-06-27T18:00:00.000Z
ID agend.: 24f21de9-f6f5-428e-b45a-ccf50eb2b364
Ação simulada: push/e-mail ao app do prestador

[NOTIFICAÇÃO → CLIENTE] Agendamento confirmado pelo barbeiro
Evento:    appointment.confirmed (7c10eec4-b808-42e3-acdb-b3ca828a452f)
Cliente:   Caio Kodato Cliente <caio victorkodatot@gmail.com>
Barbeiro:  João Barbeiro
Horário:   2026-06-27T17:30:00.000Z → 2026-06-27T18:00:00.000Z
ID agend.: 24f21de9-f6f5-428e-b45a-ccf50eb2b364
Ação simulada: push/e-mail ao app do cliente

```

O worker, em **processo separado**, consome as filas e simula notificação ao barbeiro e ao cliente.

Figura 3 — Painel RabbitMQ (exchange e bindings)



Exchange **topic** `barbertime.events` com roteamento para as filas de notificação.

Figura 4 — Painel RabbitMQ (filas)

RabbitMQ™

RabbitMQ 3.13.7

Erlang 26.2.5.16

Refreshed 2026-05-25 17:01:12

Refresh every 5 seconds

Virtual host: All

Cluster: rabbit@2b088d805602

User: barbertime

Log out

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Queues

All queues (2)

Page 1 of 1 - Filter:

Regex?

Displaying 2 items, page size up to: 100

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	barbertime.notifications.barber	classic	D	running	0	0	0	0.00/s	0.00/s	0.00/s	
/	barbertime.notifications.client	classic	D	running	0	0	0	0.00/s	0.00/s	0.00/s	

Add a new queue

HTTP API

Documentation

Tutorials

New releases

Commercial edition

Commercial support

Discussions

Discord

Plugins

GitHub

Filas duráveis em execução; mensagens consumidas após os testes (Ready = 0).

Painel: <http://localhost:15673> — usuário e senha `barbertime` .

Como reproduzir

```
docker compose up -d
npm run worker:notifications
npm run dev
```

Postman: register → available-slots → POST /appointments → login barbeiro → PATCH confirm.

Coleção: `postman/BarberTime.postman_collection.json`